

4. Security
5. Code integrity
6. High performance
7. Robustness
8. Scalability

A list of well-known open source application servers are:

1. GlassFish (available under Common Development and Distribution License and the GPL)
2. JBoss (GPL)
3. JOnAs (GPL)

A notable exception in the above list is Apache

Tomcat. It is a topic of continuous discussion among Java developers on whether this is an application server or a Web server. The classification actually depends upon how it is used. Refer www.javaworld.com/javaworld/jw-01-2008/jw-01-tomcat6.html for an interesting discussion.

In this article I am going to focus on GlassFish, which is a community-driven project started by Sun Microsystems in June 2005. I will present this article in two parts, the first of which will explain the basics about application servers and the architecture of GlassFish. In the second part, I will cover getting started with GlassFish on Linux.

The foundation of the GlassFish community

GlassFish [glassfish.dev.java.net] is a community driven project to develop an open source productive Java EE 5 application server.

The base code of the project is donated by two industry giants:

1. Sun Microsystems—Sun Java System Application Server PE 9
2. Oracle—Top Link Persistence

The architecture of GlassFish

The GlassFish community describes the design goal of GlassFish as:

1. To make an open, modular, extensible platform
2. A fast, easy, reliable application server
3. An enterprise-ready application server with massive scalability and sophisticated administration
4. Product updates and add-ons through the industry-strength Update Centre 2.0
5. Support for OSGi
6. Support for dynamic languages such as Ruby and Groovy
7. Support for Java EE 6

Let us go on to explore the overall design architecture.

A modular subsystem

In enterprise development, where large scale distributed applications are continuously running, there is a strong need for isolation between various parts of an application. This is required because if any part of a large scale distributed application crashes, the whole system should not crash. Therefore, an application server ideally should

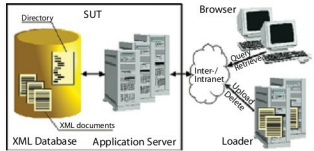


Figure 1: General architecture of an application server

be an assembly of modular sub-components.

The modular subsystem for GlassFish is based upon the Hundred KiloByte Kernel [hk2.dev.java.net]. Generally known as HK2, it is a sub-project under the GlassFish Community. If we look closely at HK2, it is based on two technologies:

1. Modules Subsystem, and
2. Component Model

Modules Subsystem is responsible for instantiating various classes to provide application functionality and the Component Model is built upon the Modules Subsystem.

Component Model works very closely with Modules Subsystem and it configures the various objects created by the Modules Subsystem – for example, bringing in other objects that are required by the newly created object, providing the newly-created objects to other existing objects, etc.

Modules management

The GlassFish wiki defines the module management system as 'a distribution artifact'. A distribution is a collection of various modules and an artifact is a file that contains the distribution. GlassFish v3 will be available in various distributions. Just as today we have various distributions of GNU/Linux, each with its own set of features, similarly, various distributions of the GlassFish application server will have various modules and distinct features. Module management depends upon Maven [<http://maven.apache.org>]—a Java project management tool.

To add your module in a distribution, you have to follow these three steps:

1. Compile your module from its source code.
2. Create a file that contains the module.
3. Add the module's identification in the distribution list.

Runtime

The GlassFish Application server is designed as a set of modules. It has no main class for its start up. Instead, it has a bootstrap module that is first loaded by the Class Loader. The bootstrap module loads other modules that are required. Also, care has been taken so that the bootstrap module does not load all the modules at the start time. But suppose the modules have been programmed such